Interpolation

Chapter 4 of Numerical Analysis

Yiping Cheng
ypcheng@bjtu.edu.cn
9-Sep-2022

School of Electronic and Information Engineering
Beijing Jiaotong University

Chapter Contents

1 Standard Polynomial Interpolation
- Lagrange's Interpolation Formula
- Newton's Interpolation Formula
- Interpolation Error

2 Spline Interpolation
- Interpolation by Cubic Splines

3 Bezier Curves
- Definition of Bezier Curves
- Properties of Bezier Curves
- Application: TrueType Fonts

Standard Polynomial Interpolation

The standard polynomial interpolation problem

Suppose that we have a table of numerical values of a function:

x	x_0	x_1	$\cdots$	x_n
y	y_0	y_1	$\cdots$	y_n

The x_i's are all distinct. Is it possible to find a simple and convenient formula that reproduces the given points exactly? One can easily think of using a polynomial to do that, although many other types of simple functions can also be used. Once a simple function p has been obtained, it can be used in place of f in many situations. For example, the integral of f could be estimated by the integral of p, and the latter should generally be easier to evaluate.

Interpolating polynomial: Lagrange form

Suppose that we wish to interpolate arbitrary functions at a set of fixed nodes $x_0, x_1, \cdots, x_n$. We first define a system of $n+1$ special polynomials of degree n known as cardinal polynomials in interpolation theory. These are denoted by $\ell_0, \ell_1, \cdots, \ell_n$ and have the property

$$\ell_i(x_j) = \delta_{ij} = \begin{cases} 1 & \text{if } i = j \\ 0 & \text{if } i \neq j \end{cases}$$

Once these are available, we can interpolate any function f by the Lagrange form of the interpolation polynomial:

$$p(x) = \sum_{i=0}^{n} \ell_i(x) f(x_i).$$

This function p is itself a polynomial of degree at most n.

It remains now only to write the formula for the cardinal polynomial ℓ_i , which is

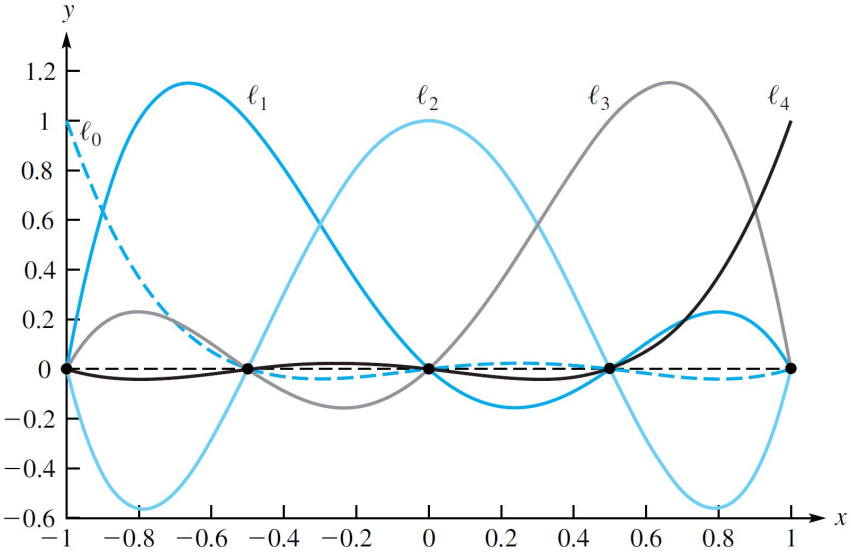
$$\ell_i(x) = \prod_{0 \le j \le n, j \ne i} \frac{x - x_j}{x_i - x_j}.$$

This formula indicates that $\ell_i(x)$ is the product of n linear factors:

$$\ell_i(x) = (\frac{x - x_0}{x_i - x_0})(\frac{x - x_1}{x_i - x_1}) \cdots (\frac{x - x_{i-1}}{x_i - x_{i-1}})(\frac{x - x_{i+1}}{x_i - x_{i+1}}) \cdots (\frac{x - x_n}{x_i - x_n})$$

(The denominators are just numbers; the variable x occurs only in the numerators.) Thus, ℓ_i is a polynomial of degree n. Notice that when $\ell_i(x)$ is evaluated at $x = x_i$, each factor in the preceding equation becomes 1. Hence, $\ell_i(x_i) = 1$. But when $\ell_i(x)$ is evaluated at any other node, say, x_j, one of the factors in the above equation will be 0, and $\ell_i(x_j) = 0$, for $i \ne j$.

The following figure shows the first few Lagrange cardinal polynomials:



Example

Write out the cardinal polynomials appropriate to the problem of interpolating the following table, and give the Lagrange form of the interpolating polynomial:

x	$\frac{1}{3}$	$\frac{1}{4}$	1
$f(x)$	2	-1	7

We have

$$\ell_0(x) = \frac{(x-\frac{1}{4})(x-1)}{(\frac{1}{3}-\frac{1}{4})(\frac{1}{3}-1)} = -18(x-\frac{1}{4})(x-1)$$

$$\ell_1(x) = \frac{(x-\frac{1}{3})(x-1)}{(\frac{1}{4}-\frac{1}{3})(\frac{1}{4}-1)} = 16(x-\frac{1}{3})(x-1)$$

$$\ell_2(x) = \frac{(x-\frac{1}{3})(x-\frac{1}{4})}{(1-\frac{1}{3})(1-\frac{1}{4})} = 2(x-\frac{1}{3})(x-\frac{1}{4})$$

Therefore, the interpolating polynomial in Lagrange's form is

$$p(x) = -36(x-\frac{1}{4})(x-1) - 16(x-\frac{1}{3})(x-1) + 14(x-\frac{1}{3})(x-\frac{1}{4}).$$

Coefficients of Lagrange cardinal polynomials

Define for variables $\{r_1, \cdots, r_n\}$ elementary symmetric functions:

$$\sigma_0(\{r_1, \cdots, r_n\}) = 1$$

$$\sigma_m(\{r_1, \cdots, r_n\}) = \sum_{1 \le j_1 < j_2 < \cdots < j_m \le n} r_{j_1} r_{j_2} \cdots r_{j_m} \text{ for } m = 1, \ldots, n$$

Let us denote

$$\ell_i(x) = \sum_{k=0}^{n} c_{i,k} x^k.$$

Then

$$c_{i,k} = \frac{(-1)^{n-k} \sigma_{n-k}(\{x_0, x_1, \cdots, x_n\} \backslash \{x_i\})}{\prod_{0 \le j \le n, j \ne i} (x_i - x_j)}.$$

This can also be viewed from another perspective. It can be derived from the definition of Lagrange cardinal polynomials that

$$\begin{bmatrix} c_{0,0} & c_{0,1} & \cdots & c_{0,n} \\ c_{1,0} & c_{1,1} & \cdots & c_{1,n} \\ \vdots & \vdots & \ddots & \vdots \\ c_{n,0} & c_{n,1} & \cdots & c_{n,n} \end{bmatrix} \begin{bmatrix} 1 & 1 & \cdots & 1 \\ x_0 & x_1 & \cdots & x_n \\ \vdots & \vdots & \ddots & \vdots \\ x_0^n & x_1^n & \cdots & x_n^n \end{bmatrix} = \boldsymbol{I}.$$

And this also implies that

$$\begin{bmatrix} 1 & 1 & \cdots & 1 \\ x_0 & x_1 & \cdots & x_n \\ \vdots & \vdots & \ddots & \vdots \\ x_0^n & x_1^n & \cdots & x_n^n \end{bmatrix} \begin{bmatrix} c_{0,0} & c_{0,1} & \cdots & c_{0,n} \\ c_{1,0} & c_{1,1} & \cdots & c_{1,n} \\ \vdots & \vdots & \ddots & \vdots \\ c_{n,0} & c_{n,1} & \cdots & c_{n,n} \end{bmatrix} = \boldsymbol{I}.$$

Introducing Newton's interpolation formula

The Lagrange interpolation formula proves the existence of an interpolating polynomial for any table of values. There is another constructive way of proving this fact, and it leads to a different formula.

Suppose that $p_k(x_i) = y_i$ for $0 \le i \le k$. Then $p_{k+1}(x) = p_k(x) + c\prod_{i=0}^{k}(x - x_i)$ also has this property. And we choose a suitable c such that $p_{k+1}(x_{k+1}) = y_{k+1}$. That is,

$$p_k(x_{k+1}) + c(x_{k+1} - x_0)(x_{k+1} - x_1)\cdots(x_{k+1} - x_k) = y_{k+1}.$$

The proper value of c can be obtained from this equation because none of the factors $x_{k+1} - x_i$, for $0 \le i \le k$, can be zero. Remember our original assumption that the x_i's are all distinct.

Interpolating polynomial: Newton form

We thus obtain the following form of interpolating polynomial:

$$p_n(x) = \sum_{k=0}^{n} a_k \prod_{i=0}^{k-1} (x - x_i).$$

It is called Newton form. It can not be overemphasized that the Newton and Lagrange forms are just two different derivations for precisely the same polynomial. The Newton form has the advantage of easy extensibility to accommodate additional data points.

Example

Find the Newton form interpolating polynomial of this table:

x	0	1	-1
y	5	3	15

Later, we will develop a better algorithm for constructing the Newton interpolating polynomial. Nevertheless, the method just explained is a systematic one and involves very little computation. An important feature to notice is that each new polynomial in the algorithm is obtained from its predecessor by adding a new term. Thus, at the end, the final polynomial exhibits all the previous polynomials as constituents.

Newton form in nested form

Let $n = 4$, then

$$p_4(x)=a_0+a_1(x-x_0)+a_2(x-x_0)(x-x_1)+a_3(x-x_0)(x-x_1)(x-x_2)+a_4(x-x_0)(x-x_1)(x-x_2)(x-x_3)$$

$$= (((a_4(x - x_3) + a_3)(x - x_2) + a_2)(x - x_1) + a_1)(x - x_0) + a_0.$$

In evaluating $p(x)$ for a given numerical value of x,we naturally start with the innermost parentheses, forming successively the following quantities:

$$\begin{aligned} v_0 &= a_n \\ v_1 &= v_0(x - x_{n-1}) + a_{n-1} \\ v_2 &= v_1(x - x_{n-2}) + a_{n-2} \\ &\vdots \\ v_n &= v_{n-1}(x - x_0) + a_0 \end{aligned}$$

A new formula for Newton form coefficients

Theorem

Suppose for all $0 \le k \le n$,

$$a_k = \sum_{i=0}^{k} \frac{y_i}{\prod\limits_{0 \le j \le k, j \ne i} (x_i - x_j)}.$$

Then for all $0 \le m \le n$,

$$\sum_{k=0}^{n} a_k \prod_{l=0}^{k-1} (x_m - x_l) = y_m.$$

Proof.

We only need to show $\sum_{k=0}^{n} a_k \prod_{l=0}^{k-1} (x_n - x_l) = y_n$. Once we have done that, we immediately have $\sum\limits_{k=0}^{n} a_k \prod_{l=0}^{k-1} (x_m - x_l) = \underbrace{\sum_{k=0}^{m} a_k \prod_{l=0}^{k-1} (x_m - x_l)}_{=y_m} + \sum\limits_{k=m+1}^{n} a_k \underbrace{\prod_{l=0}^{k-1} (x_m - x_l)}_{=0} = y_m.$ □

Proof.

$$\sum_{k=0}^{n} a_k \prod_{l=0}^{k-1}(x_n - x_l) = \sum_{k=0}^{n}\sum_{i=0}^{k} \frac{y_i}{\prod\limits_{0\le j\le k, j\ne i}(x_i - x_j)} \prod_{l=0}^{k-1}(x_n - x_l)$$

$$= \sum_{i=0}^{n}\sum_{k=i}^{n} \frac{y_i}{\prod\limits_{0\le j\le k, j\ne i}(x_i - x_j)} \prod_{l=0}^{k-1}(x_n - x_l) = \sum_{i=0}^{n} y_i \sum_{k=i}^{n} \frac{\prod\limits_{l=0}^{k-1}(x_n - x_l)}{\prod\limits_{0\le j\le k, j\ne i}(x_i - x_j)}$$

$$= \sum_{i=0}^{n} y_i \frac{\prod\limits_{l=0}^{i-1}(x_n - x_l)}{\prod\limits_{0\le j\le n, j\ne i}(x_i - x_j)} \sum_{k=i}^{n}\left(\prod_{l=i}^{k-1}(x_n - x_l)\prod_{j=k+1}^{n}(x_i - x_j)\right)$$

If $i = n$, then

$$\frac{\prod\limits_{l=0}^{i-1}(x_n - x_l)}{\prod\limits_{0\le j\le n, j\ne i}(x_i - x_j)} \sum_{k=i}^{n}\left(\prod_{l=i}^{k-1}(x_n - x_l)\prod_{j=k+1}^{n}(x_i - x_j)\right) = 1.$$

Proof.

If $i < n$, we claim that

$$\sum_{k=i}^{n}\left(\prod_{l=i}^{k-1}(x_n - x_l)\prod_{j=k+1}^{n}(x_i - x_j)\right) = 0.$$

This is established by showing the following lemma: For $0 \le d \le n-i$,

$$\sum_{k=n-d}^{n}\left(\prod_{l=i}^{k-1}(x_n - x_l)\prod_{j=k+1}^{n}(x_i - x_j)\right) = \prod_{l=i}^{n-d-1}(x_n - x_l)\prod_{j=n-d}^{n-1}(x_i - x_j).$$

When $d = n-i$, it is evident that this lemma leads to our claim. Now, we will prove this lemma.
Base case: $d = 0$. We have

$$\sum_{k=n-d}^{n}\left(\prod_{l=i}^{k-1}(x_n - x_l)\prod_{j=k+1}^{n}(x_i - x_j)\right) = \prod_{l=i}^{n-1}(x_n - x_l) = \prod_{l=i}^{n-d-1}(x_n - x_l)\prod_{j=n-d}^{n-1}(x_i - x_j).$$

□

Proof.

Inductive step: Let $1 \le d \le n-i$, and suppose that the lemma holds for $d-1$, that is

$$\sum_{k=n-d+1}^{n}\left(\prod_{l=i}^{k-1}(x_n - x_l)\prod_{j=k+1}^{n}(x_i - x_j)\right) = \prod_{l=i}^{n-d}(x_n - x_l)\prod_{j=n-d+1}^{n-1}(x_i - x_j)$$

$$= \prod_{l=i}^{n-d-1}(x_n - x_l)\cdot(x_n - x_{n-d})\cdot\prod_{j=n-d+1}^{n-1}(x_i - x_j).$$

And $\displaystyle\prod_{l=i}^{n-d-1}(x_n - x_l)\prod_{j=n-d+1}^{n}(x_i - x_j) = \prod_{l=i}^{n-d-1}(x_n - x_l)\cdot(x_i - x_n)\cdot\prod_{j=n-d+1}^{n-1}(x_i - x_j).$

Adding the last two expressions, we obtain

$$\sum_{k=n-d}^{n}\left(\prod_{l=i}^{k-1}(x_n - x_l)\prod_{j=k+1}^{n}(x_i - x_j)\right) = \prod_{l=i}^{n-d-1}(x_n - x_l)\cdot(x_i - x_{n-d})\cdot\prod_{j=n-d+1}^{n-1}(x_i - x_j)$$

$$= \prod_{l=i}^{n-d-1}(x_n - x_l)\prod_{j=n-d+1}^{n-1}(x_i - x_j).$$

Defining divided differences

Definition (Divided difference)

The divided difference of function f on a set of distinct points $\{x_0, x_1, \cdots, x_k\}$, denoted by $f[x_0, x_1, \cdots, x_k]$, is defined by

$$f[x_0, x_1, \cdots, x_k] = \sum_{i=0}^{k} \frac{f(x_i)}{\prod_{0 \le j \le k, j \ne i} (x_i - x_j)}.$$

Some remarks:

- The divided difference could also be defined for a set of input-output data pairs. But the convention used in the literature is that it is defined for functions.
- From the definition it can be seen that the divided difference is symmetric with respect to the points, i.e., if f is fixed, then it only depends on the **set** of points, but not on the order between them.
- Newton's interpolating polynomial now takes the form:

$$p_n(x) = \sum_{k=0}^{n} \left(f[x_0, x_1, \cdots, x_k] \prod_{j=0}^{k-1} (x - x_j) \right).$$

Recursive property of divided differences

It is not difficult to verify

$$
\begin{aligned}
f[x_0] &= f(x_0) \\
f[x_0, x_1] &= \frac{f(x_1) - f(x_0)}{x_1 - x_0} \\
f[x_0, x_1, x_2] &= \frac{\frac{f(x_2)-f(x_1)}{x_2-x_1} - \frac{f(x_1)-f(x_0)}{x_1-x_0}}{x_2 - x_0} \\
&\vdots
\end{aligned}
$$

This is not a coincidence. And generally we have

Theorem (recursive property of divided differences)

The divided differences obey the formula

$$f[x_0, x_1, \cdots, x_k] = \frac{f[x_1, x_2, \cdots, x_k] - f[x_0, x_1, \cdots, x_{k-1}]}{x_k - x_0}.$$

Proof.

$$\frac{f[x_1,x_2,\cdots,x_k]-f[x_0,x_1,\cdots,x_{k-1}]}{x_k-x_0}=\frac{\sum_{i=1}^{k}\frac{f(x_i)}{\prod\limits_{1\le j\le k,j\ne i}(x_i-x_j)}-\sum_{i=0}^{k-1}\frac{f(x_i)}{\prod\limits_{0\le j\le k-1,j\ne i}(x_i-x_j)}}{x_k-x_0}$$

$$=\frac{\frac{f(x_k)}{\prod\limits_{1\le j\le k-1}(x_k-x_j)}-\frac{f(x_0)}{\prod\limits_{1\le j\le k-1}(x_0-x_j)}+\sum_{i=1}^{k-1}\frac{f(x_i)}{\prod\limits_{1\le j\le k,j\ne i}(x_i-x_j)}-\sum_{i=1}^{k-1}\frac{f(x_i)}{\prod\limits_{0\le j\le k-1,j\ne i}(x_i-x_j)}}{x_k-x_0}$$

$$=\frac{\frac{f(x_k)}{\prod\limits_{1\le j\le k-1}(x_k-x_j)}-\frac{f(x_0)}{\prod\limits_{1\le j\le k-1}(x_0-x_j)}+\sum_{i=1}^{k-1}\frac{f(x_i)(x_i-x_0)}{\prod\limits_{0\le j\le k,j\ne i}(x_i-x_j)}-\sum_{i=1}^{k-1}\frac{f(x_i)(x_i-x_k)}{\prod\limits_{0\le j\le k,j\ne i}(x_i-x_j)}}{x_k-x_0}$$

$$=\frac{f(x_k)}{\prod\limits_{0\le j\le k-1}(x_k-x_j)}+\frac{f(x_0)}{\prod\limits_{1\le j\le k}(x_0-x_j)}+\sum_{i=1}^{k-1}\frac{f(x_i)}{\prod\limits_{0\le j\le k,j\ne i}(x_i-x_j)}$$

$$=f[x_0,x_1,\cdots,x_k].$$

□

Example

Construct a divided-difference diagram for the function f given in the following table, and write out the Newton form of the interpolating polynomial.

x	1	$\frac{3}{2}$	0	2
$f(x)$	3	$\frac{13}{4}$	3	$\frac{5}{3}$

Solution: The first entry is $f[x_0, x_1] = (\frac{13}{4} - 3)(\frac{3}{2} - 1) = \frac{1}{2}$. After completion of column 3, the first entry in column 4 is

$$f[x_0, x_1, x_2] = \frac{f[x_1, x_2] - f[x_0, x_1]}{x_2 - x_0} = \frac{\frac{1}{6} - \frac{1}{2}}{0 - 1} = \frac{1}{3}.$$

The complete diagram is

x	$f[x_0]$	$f[x_0, x_1]$	$f[x_0, x_1, x_2]$	$f[x_0, x_1, x_2, x_3]$
1	3			
$\frac{3}{2}$	$\frac{13}{4}$	$\frac{1}{2}$		
0	3	$\frac{1}{6}$	$\frac{1}{3}$	
2	$\frac{5}{3}$	$-\frac{2}{3}$	$-\frac{5}{3}$	-2

Thus, we obtain $p_3(x) = 3 + \frac{1}{2}(x - 1) + \frac{1}{3}(x - 1)(x - \frac{3}{2}) - 2(x - 1)(x - \frac{3}{2})x$.

To be completed

Spline Interpolation

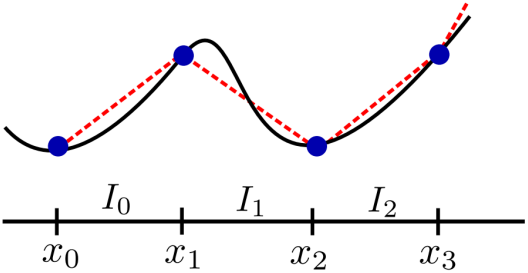
The previous sections concerned the approximation of arbitrary functions on closed intervals using a single polynomial. However, high-degree polynomials can oscillate erratically, that is, a minor fluctuation over a small portion of the interval can induce large fluctuations over the entire range. The Runge phenomenon shows that.
An alternative approach is to divide the approximation interval into a collection of subintervals and construct a (generally) different approximating polynomial on each subinterval. This is called piecewise-polynomial approximation.

The most common piecewise-polynomial approximation uses cubic polynomials between each successive pair of nodes and is called cubic spline interpolation.

Cubic spline interpolation

Given $(x_0, y_0), (x_1, y_1), \cdots, (x_n, y_n)$, fit cubic splines through the data. The splines are given by

$$S(x) = \begin{cases} S_1(x) = a_1x^3 + b_1x^2 + c_1x + d_1 & \text{if } x_0 \le x \le x_1 \\ S_2(x) = a_2x^3 + b_2x^2 + c_2x + d_2 & \text{if } x_1 \le x \le x_2 \\ \vdots & \vdots \\ S_n(x) = a_nx^3 + b_nx^2 + c_nx + d_n & \text{if } x_{n-1} \le x \le x_n \end{cases}$$



Required properties of the cubic spline

1. For each $1 \le i \le n$, $S_i(x_{i-1}) = y_{i-1}$, $S_i(x_i) = y_i$.
2. For each $1 \le i \le n-1$, $S_i'(x_i-) = S_{i+1}'(x_i+)$.
3. For each $1 \le i \le n-1$, $S_i''(x_i-) = S_{i+1}''(x_i+)$.

Property 1 guarantees that the spline $S(x)$ interpolates the data points. Property 2 forces the slopes of neighboring parts of the spline to agree where they meet, and Property 3 does the same for the curvature, represented by the second derivative.

Constructing a spline from a set of data points means finding the coefficients a_i, b_i, c_i, d_i that make Properties 1–3 hold. Before we discuss how to determine the unknown coefficients of the spline, let us count the number of conditions imposed by the definition. Property 1 consists of $2n$ equations that must be satisfied by the coefficients, which we consider as unknowns. Each of Properties 2 and 3 add $n-1$ additional equations, for a total of $2n + 2(n-1) = 4n-2$ independent equations to be satisfied.

Natural cubic spline

Unless there are inconsistent equations in the system (and there are not), the system of equations is underdetermined and so has infinitely many solutions. In other words, there are infinitely many cubic splines passing through the data points.

Users of splines normally exploit the shortage of equations by adding two extra to the $4n-2$ equations to arrive at a system of $4n$ equations in $4n$ unknowns. Aside from allowing the user to constrain the spline to given specifications, narrowing the field to a single solution simplifies computing and describing the result.

Consider adding to Properties 1–3 the following constraints:

Natural spline: $S_1''(x_0) = 0$ and $S_n''(x_n) = 0$.

Construction of the cubic spline

In this slide we will derive the parameters of the segments of the cubic spline. For each $0 \le i \le n$, we denote

$$\begin{aligned} h_i &= x_i - x_{i-1} \text{ for } i \ge 1 \\ M_i &= S''(x_i) \end{aligned}$$

Since each segment is a cubic function, $S''(x)$ is linear in each segment. Thus for each $1 \le i \le n$,

$$S''(x) = M_i \frac{x - x_{i-1}}{h_i} + M_{i-1} \frac{x_i - x}{h_i} \text{ for } x \in [x_{i-1}, x_i].$$

Integrating it twice we get

$$S(x) = \frac{M_i}{6h_i}(x - x_{i-1})^3 + \frac{M_{i-1}}{6h_i}(x_i - x)^3 + \alpha_i(x - x_{i-1}) + \beta_i(x_i - x) \text{ for } x \in [x_{i-1}, x_i].$$

Since $S(x_{i-1}) = y_{i-1}$ and $S(x_i) = y_i$, we obtain $\alpha_i = \frac{y_i}{h_i} - \frac{M_i}{6}h_i$ and $\beta_i = \frac{y_{i-1}}{h_i} - \frac{M_{i-1}}{6}h_i$, and for $x \in [x_{i-1}, x_i]$,

$$S(x) = \frac{M_i}{6h_i}(x - x_{i-1})^3 + \frac{M_{i-1}}{6h_i}(x_i - x)^3 + (\frac{y_i}{h_i} - \frac{M_i}{6}h_i)(x - x_{i-1}) + (\frac{y_{i-1}}{h_i} - \frac{M_{i-1}}{6}h_i)(x_i - x).$$

Therefore for $x \in [x_{i-1}, x_i]$,

$$S'(x) = \frac{M_i}{2h_i}(x - x_{i-1})^2 - \frac{M_{i-1}}{2h_i}(x_i - x)^2 + \frac{y_i - y_{i-1}}{h_i} + \frac{M_{i-1} - M_i}{6}h_i.$$

As a result, for each $1 \le i \le n-1$,

$$\begin{aligned} S'(x_i-) &= \frac{h_i}{6}M_{i-1} + \frac{h_i}{3}M_i + \frac{y_i - y_{i-1}}{h_i} \\ S'(x_i+) &= -\frac{h_{i+1}}{3}M_i - \frac{h_{i+1}}{6}M_{i+1} + \frac{y_{i+1} - y_i}{h_{i+1}} \end{aligned}$$

As required, $S'(x_i-) = S'(x_i+)$, thus

$$\frac{h_i}{6}M_{i-1} + \frac{h_i + h_{i+1}}{3}M_i + \frac{h_{i+1}}{6}M_{i+1} = \frac{y_{i+1} - y_i}{h_{i+1}} - \frac{y_i - y_{i-1}}{h_i},$$

$$\frac{h_i}{h_i + h_{i+1}}M_{i-1} + 2M_i + \frac{h_{i+1}}{h_i + h_{i+1}}M_{i+1} = \frac{6}{h_i + h_{i+1}}\left(\frac{y_{i+1} - y_i}{h_{i+1}} - \frac{y_i - y_{i-1}}{h_i}\right)$$

So the parameters $M_1, M_2, \cdots, M_{n-2}, M_{n-1}$ can be obtained by solving this tri-diagonal matrix equation:

$$\begin{bmatrix} 2 & \frac{h_2}{h_1+h_2} & & & \\ \frac{h_2}{h_2+h_3} & 2 & \frac{h_3}{h_2+h_3} & & \\ & \ddots & \ddots & \ddots & \\ & & \frac{h_{n-2}}{h_{n-2}+h_{n-1}} & 2 & \frac{h_{n-1}}{h_{n-2}+h_{n-1}} \\ & & & \frac{h_{n-1}}{h_{n-1}+h_n} & 2 \end{bmatrix} \begin{bmatrix} M_1 \\ M_2 \\ \vdots \\ M_{n-2} \\ M_{n-1} \end{bmatrix} = \begin{bmatrix} \frac{6}{h_1+h_2}\left(\frac{y_2-y_1}{h_2} - \frac{y_1-y_0}{h_1}\right) \\ \frac{6}{h_2+h_3}\left(\frac{y_3-y_2}{h_3} - \frac{y_2-y_1}{h_2}\right) \\ \vdots \\ \frac{6}{h_{n-2}+h_{n-1}}\left(\frac{y_{n-1}-y_{n-2}}{h_{n-1}} - \frac{y_{n-2}-y_{n-3}}{h_{n-2}}\right) \\ \frac{6}{h_{n-1}+h_n}\left(\frac{y_n-y_{n-1}}{h_n} - \frac{y_{n-1}-y_{n-2}}{h_{n-1}}\right) \end{bmatrix}$$

Bezier Curves

Linear Bezier curves

Definition (Linear Bezier curve)

Given two control points $\boldsymbol{b}_0$ and $\boldsymbol{b}_1$, we define the linear Bezier curve to be the curve parametrized by:

$$\boldsymbol{b}(t) = (1-t)\boldsymbol{b}_0 + t\boldsymbol{b}_1 \text{ for } t \in [0,1].$$

It is clear that this leads to a straight line from $\boldsymbol{b}_0$ to $\boldsymbol{b}_1$.

Quadratic Bezier curves

Definition (Quadratic Bezier curve)

Given three control points $\boldsymbol{b}_0$, $\boldsymbol{b}_1$ and $\boldsymbol{b}_2$, we define the quadratic Bezier curve to be the curve parametrized by:

$$\boldsymbol{b}(t) = (1-t)^2\boldsymbol{b}_0 + 2t(1-t)\boldsymbol{b}_1 + t^2\boldsymbol{b}_2 \text{ for } t \in [0,1].$$

Notice what is happening here. At $t = 0$ the vector $\boldsymbol{b}_0$ is the only thing accounted for and at $t = 1$ the vector $\boldsymbol{b}_2$ is the only thing accounted for. For intermediate values of t the function $\boldsymbol{b}(t)$ assigns a shifting weight to the various control points.

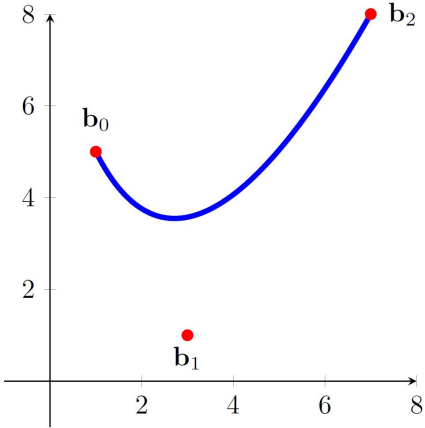
We see that the coefficients start weighted to that of $\boldsymbol{b}_0$ and shift at the end to $\boldsymbol{b}_2$. They never get completely to $\boldsymbol{b}_1$ although they tend there in the middle. Let's look at a simple example in $\mathbb{R}^2$.

Example

Given the three control points: $\boldsymbol{b}_0 = [1; 5]$, $\boldsymbol{b}_1 = [3; 1]$, $\boldsymbol{b}_2 = [7; 8]$. The quadratic Bezier curve is given by:

$$\boldsymbol{b}(t) = (1-t)^2[1;5] + 2t(1-t)[3;1] + t^2[7;8]$$

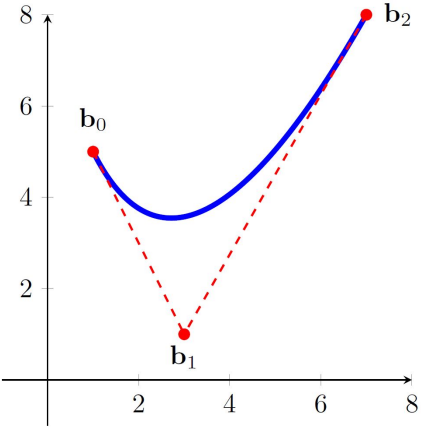
$$= [2t^2 + 4t + 1; 11t^2 - 8t + 5].$$



Notice the following:

- The curve passes through the two end control points. It is clear that this will always be the case since $\boldsymbol{b}(0) = \boldsymbol{b}_0$ and $\boldsymbol{b}(1) = \boldsymbol{b}_2$.
- The curve does not (necessarily) pass through the middle control point.
- The curve exits $\boldsymbol{b}_0$ in the direction of $\boldsymbol{b}_1$.
- The curve enters $\boldsymbol{b}_2$ from the direction of $\boldsymbol{b}_1$.

The final two notes can be clarified by drawing the lines tangent to the curve at the start and end control points:



Theorem

For control points $\boldsymbol{b}_0$, $\boldsymbol{b}_1$ and $\boldsymbol{b}_2$ the lines tangent to the quadratic Bezier curve at $t = 0$ and $t = 1$ both intersect $\boldsymbol{b}_1$.

Proof.

$$\boldsymbol{b}(t) = (1-t)^2\boldsymbol{b}_0 + 2t(1-t)\boldsymbol{b}_1 + t^2\boldsymbol{b}_2$$

$$\boldsymbol{b}'(t) = 2(t-1)\boldsymbol{b}_0 + (2-4t)\boldsymbol{b}_1 + 2t\boldsymbol{b}_2$$

$$\boldsymbol{b}'(0) = -2\boldsymbol{b}_0 + 2\boldsymbol{b}_1 = 2(\boldsymbol{b}_1 - \boldsymbol{b}_0)$$

Since this is a multiple of $\boldsymbol{b}_1 - \boldsymbol{b}_0$ it follows that the vector tangent to the curve at $\boldsymbol{b}_0$ points from $\boldsymbol{b}_0$ towards $\boldsymbol{b}_1$. The proof for the other endpoint is similar. □

Cubic Bezier curves

Definition (Cubic Bezier curve)

Given four control points $\boldsymbol{b}_0$, $\boldsymbol{b}_1$, $\boldsymbol{b}_2$, and $\boldsymbol{b}_3$, we define the cubic Bezier curve to be the curve parametrized by:

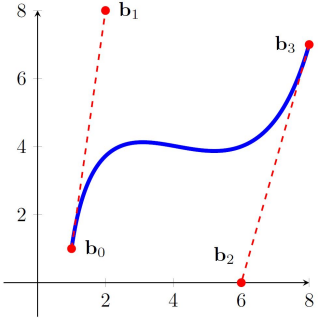
$$\boldsymbol{b}(t) = (1-t)^3\boldsymbol{b}_0 + 3t(1-t)^2\boldsymbol{b}_1 + 3t^2(1-t)\boldsymbol{b}_2 + t^3\boldsymbol{b}_3 \text{ for } t \in [0,1].$$

Example

Given the four control points: $\boldsymbol{b}_0 = [1; 1]$, $\boldsymbol{b}_1 = [2; 8]$, $\boldsymbol{b}_2 = [6; 0]$, $\boldsymbol{b}_3 = [8; 7]$. The cubic Bezier curve is given by:

$$\begin{aligned}\boldsymbol{b}(t) &= (1-t)^3[1;1] + 3t(1-t)^2[2;8] + 3t^2(1-t)[6;0] + t^3[8;7] \\ &= [-5t^3 + 9t^2 + 3t + 1; 30t^3 - 45t2 + 21t + 1].\end{aligned}$$

We see similar behavior as the quadratic in that the tangent lines at the start and end points intersubsection the point directly after and before them respectively.



General Bezier curves

Definition (General Bezier curve)

Given $n+1$ control points $\boldsymbol{b}_0$, $\boldsymbol{b}_1$, $\cdots$, and $\boldsymbol{b}_n$, we define the degree n Bezier curve to be the curve parametrized by:

$$\boldsymbol{b}(t) = \sum_{i=0}^{n} \binom{n}{i} t^i (1-t)^{n-i} \boldsymbol{b}_i \text{ for } t \in [0,1].$$

Properties of Bezier curves

Bezier curves have several properties, some of which we've seen. We present them here as a series of theorems.

Theorem

The coefficients of $\boldsymbol{b}_0, \cdots, \boldsymbol{b}_n$ add to 1.

Proof.

By the Binomial Theorem we have:

$$\sum_{i=0}^{n} \binom{n}{i} t^i (1-t)^{n-i} = (t + (1-t))^n = 1^n = 1.$$

□

Theorem

Given $n+1$ control points $\boldsymbol{b}_0, \cdots, \boldsymbol{b}_n$, the line tangent to $\boldsymbol{b}(t)$ at $\boldsymbol{b}_0$ passes through $\boldsymbol{b}_1$ and the line tangent to $\boldsymbol{b}(t)$ at $\boldsymbol{b}_n$ passes through $\boldsymbol{b}_{n-1}$.

Proof.

$$\boldsymbol{b}(t) = \sum_{i=0}^{n} \binom{n}{i} t^i (1-t)^{n-i} \boldsymbol{b}_i$$

$$\boldsymbol{b}'(t) = -n(1-t)^{n-1}\boldsymbol{b}_0 + nt^{n-1}\boldsymbol{b}_n + \sum_{i=1}^{n-1} \binom{n}{i} [it^{i-1}(1-t)^{n-i} - t^i(n-i)(1-t)^{n-i-1}]\boldsymbol{b}_i$$

$$\boldsymbol{b}'(0) = n(\boldsymbol{b}_1 - \boldsymbol{b}_0)$$

$$\boldsymbol{b}'(1) = n(\boldsymbol{b}_n - \boldsymbol{b}_{n-1})$$

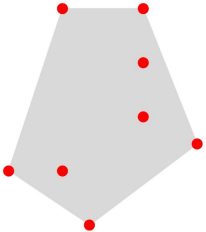
□

Definition

Given a set of $n+1$ points $\boldsymbol{b}_0, \cdots, \boldsymbol{b}_n$, the convex hull of the points is defined as:

$$\mathrm{CH}(\boldsymbol{b}_0, \cdots, \boldsymbol{b}_n) = \left\{ a_0\boldsymbol{b}_0 + \cdots + a_n\boldsymbol{b}_n | a_i \geq 0 \text{ for each } i, \sum_{i=0}^{n} a_i = 1 \right\}.$$

Visually speaking the convex hull can be pictured by stretching an elastic band so that it is as small as possible and still contains all the control points. For example here is the convex hull for a set of eight points:



Theorem

For all t, $b(t)$ lies within the convex hull of the control points.

Proof.

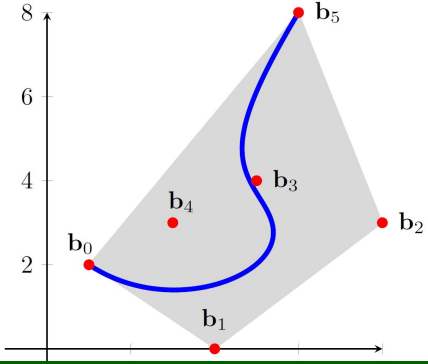
Since $b(t)$ is a subset of CH the result follows. □

Example

Consider the six control points: $\boldsymbol{b}_0 = [1;2]$, $\boldsymbol{b}_1 = [4;0]$, $\boldsymbol{b}_2 = [8;3]$, $\boldsymbol{b}_3 = [5;4]$, $\boldsymbol{b}_4 = [3;3]$, $\boldsymbol{b}_5 = [6;8]$. The degree 5 Bezier curve is:

$$b(t) = [-20t^5 + 80t^4 - 80t^3 + 10t^2 + 15t + 1; t^5 + 35t^4 - 70t^3 + 50t^2 - 10t + 2].$$

Here is a plot of the curve along with the six control points and the convex hull.



Theorem

For any linear transformation T the Bezier curve constructed using the $T(\boldsymbol{b}_i)$ is equivalent to the Bezier curve constructed using the $\boldsymbol{b}_i$ and then transformed using T.

Proof.

This follows from the fact that the Bezier curve is constructed as a linear combination of the $\boldsymbol{b}_i$ and the fact that the transformation is linear. □

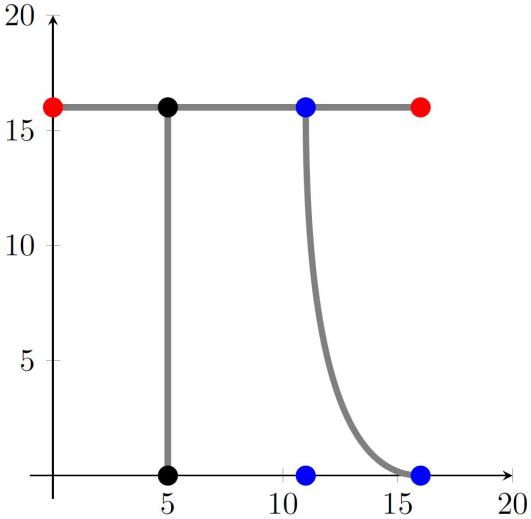
Font specification

TrueType fonts are created using Bezier curives in that each letter is created by a set of linear and quadratic Bezier curves.

Example

Suppose we wished to create a version of the Greek letter π. We might use a horizontal bar for the top, a vertical bar for the left lower part. and a vertical bar which curves to the right for the right lower part. We might create each of these as follows in a 16×16 grid:
Horizontal bar: $[0; 16]$ and $[16; 16]$
Lower left: $[5; 0]$ and $[5; 16]$
Lower right: $[11; 16]$ and $[11; 0]$ and $(16; 0]$.
If we create the Bezier curves for each of these we get the following.
Horizontal bar: $[16t; 16]$
Lower left: $[5; 16t]$
Lower right: $[5t^2 + 11; 16t^2 - 32t + 16]$

Here is the result. The control points have been drawn in red, black and blue for each of the horizontal, lower left and lower right respectively.

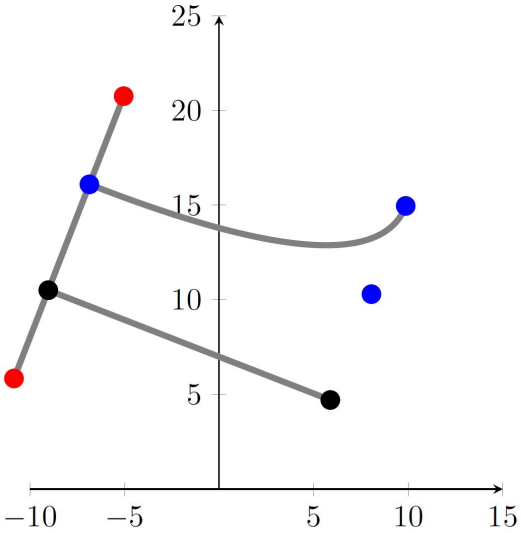


TrueType font transformations

Because Bezier curves may be transformed by transforming the control points, this allows us to translate, rotate, reflect, and scale by doing these things to the control points.

Example

Suppose we wish to rotate our letter π by 1.2 radians about the point $[2; 3]$. To do this we simply rotate the points and recreate the curves. To rotate the points we multiply by the transformation $T(2; 3)R(1.2)T(-2, -3)$.

The resulting rotated points are:
Horizontal bar: $[-10.84; 5.847]$ and $[-5.043; 20.76]$
Lower left: $[5.883; 4.709]$ and $[-9.029; 10.51]$
Lower right: $[-6.855; 16.1]$ and $[8.057; 10.3]$ and $[9.869; 14.96]$
If we create the Bezier curves for each of these we get the following.
Horizontal bar: $[5.798t - 10.84; 14.91t + 5.847]$
Lower left: $[5.83 - 14.91t; 5.798t + 4.709]$
Lower right: $[-13.1t^2 + 29.83t - 6.855; 10.46t^2 - 11.6t + 16.1]$

Here is the picture: